Causal RCA Toolbox

Quick-Start Guide

A screenshot walkthrough of the six stages

Companion to the *Operator and Engineer Manual*

Version 1.0

August 2026

How to use this guide

This is the short version. Each stage gets a screenshot, a numbered tour of the controls, and the two or three decisions that actually change your result. Where a choice needs justification — which discovery method, how to read a metric, what the tool can and cannot prove — the *Operator and Engineer Manual* is the authority, and this guide points to it.

Every screenshot is taken from the built-in synthetic demonstration dataset, so you can reproduce each one exactly. The numbered markers ❶ on the screenshots correspond to the numbered list directly beneath them.

STAGE 1

Data & Scenario

Get the data in, and mark which stretch is normal and which contains the fault.

Everything the toolbox does downstream is a *comparison* between two periods: how each sensor behaves during the fault versus how it behaves normally. Stage 1 is where you supply the data and draw that line. It takes about thirty seconds on the demo dataset.

The 30-second path

Load synthetic demo → check the auto-filled windows against the preview → **Save scenario** →. When the line under the button turns green, move to Stage 2.

The empty page

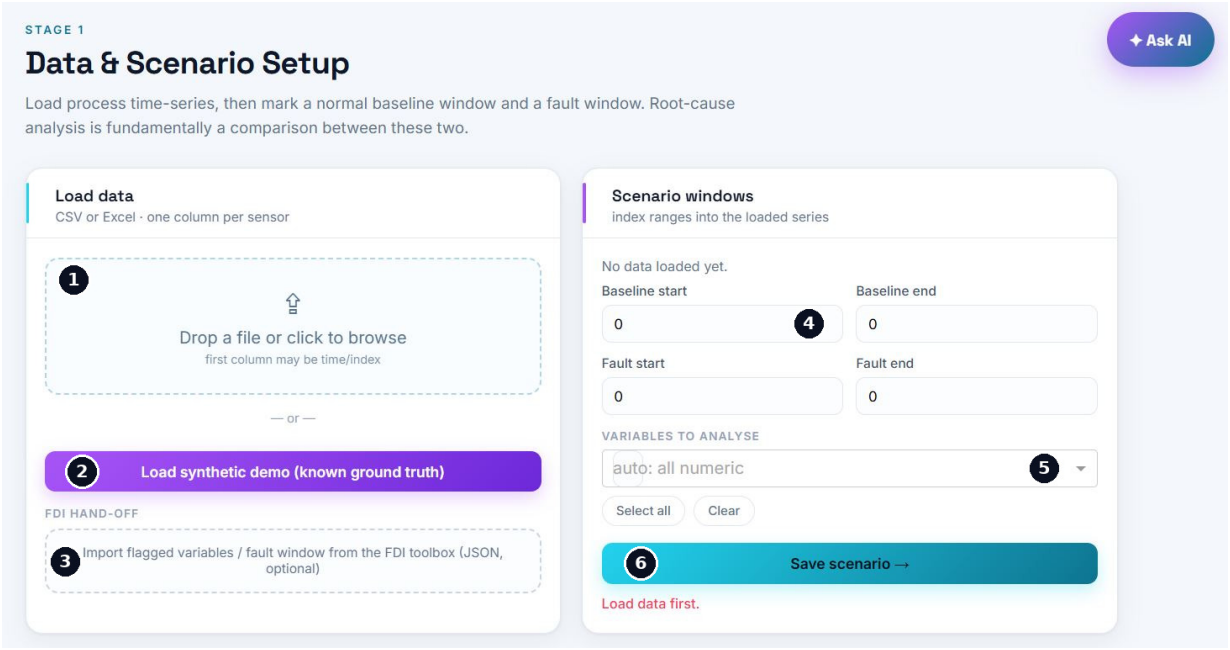

Figure 1.1: Stage 1 before any data is loaded. The left card brings data in; the right card defines the scenario.

1. **Drop zone** — CSV or Excel, one column per sensor tag, one row per time step. A leading column named `time`, `index` or `t` is used as the time axis and is not offered as a variable.

2. **Load synthetic demo** — generates 1 500 samples of five tags with a known structure: the chain `T1`→`F1`→`L1` →`P1`→`A1` plus a shortcut `T1`→`P1`, with a step disturbance entering the root `T1` at sample 900. Because the true structure is known, this is the only dataset on which the structural accuracy metrics of Stage 6 are meaningful.

3. **FDI hand-off** (optional) — imports the JSON exported by the FDI Analytics Toolbox, carrying the flagged variables and the detected fault window across so you do not retype them.

4. **Scenario windows** — four *row indices* into the loaded series, not timestamps. They fill in automatically when data loads: baseline across the first 40 % of the record, fault from 55 % to the last sample. These defaults are derived from the record length alone — no detection is involved — so always check them against the preview before saving.

5 **Variables to analyse** — leave empty to use every numeric column, or pick a subset. Fewer, well-chosen tags generally produce a clearer and more reliable graph than including everything; irrelevant sensors add spurious links and cost time.

6 **Save scenario** — nothing is stored until you press this. The status line beneath turns green on success, or red (*"Load data first."*) if there is no data yet.

The ★ **Ask AI** button in the top-right corner is present on every page and is entirely optional; see *The AI Assistant* at the end of this guide.

After loading

Figure 1.2: The same page after **Load synthetic demo** and **Save scenario**. The preview strip appears beneath the two cards.

1 **Variable chips** — every numeric column is selected by default and listed as a removable chip. Drop one with its ×.

2 **Select all** / **Clear** — shortcuts for the chip list.

3 **Confirmation line** — read it back rather than trusting the boxes: *5 vars · baseline [0:600] · fault [825:1499]*. This is what the rest of the toolbox will use.

4 **Baseline band** (cyan) — the stretch you have declared normal. Here it covers samples 0–600, where all five signals sit flat around zero.

5 **Fault band** (red) — the stretch containing the disturbance. The step is plainly visible just before sample 900, so the default start of 825 opens the window slightly early, which is exactly what you want. Note that the preview draws at most the first six variables; your selection is unaffected.

Setting the two windows

- Make the baseline long enough to contain normal variability — a few hundred samples at minimum — and representative of the operating point you care about.

- Start the fault window *at, or slightly before*, the first visible deviation. Starting late truncates the earliest evidence, which is precisely what the root-cause ranking relies on.
- The windows must not overlap. The application does not enforce this; keep **Fault start** greater than **Baseline end**.

If you change something later

Downstream stages read the *saved* scenario, not the boxes on screen. After editing a window or the variable list, press **Save scenario** again and re-run Stage 2 — otherwise you will be analysing the previous configuration.

If something looks wrong

What you see	What to do
Red *"Load data first."*	Data was never loaded, or the load failed. Load the demo or re-drop the file.
Variables to analyse stays empty	No column was read as numeric. Check the delimiter, the header row, and that numbers are not stored as text with thousands separators or units.
Preview shows fewer traces than expected	The preview is capped at the first six variables by design. The full selection is still analysed.
Fault band sits on flat, normal-looking signal	The auto-filled indices do not match this record. Read the sample number of the first deviation off the preview axis and type it into **Fault start**.

Before you move on

☐ The confirmation line is green and lists the variable count and both windows.

☐ The cyan band covers only quiet, normal signal.

☐ The red band covers the disturbance, opening at or just before its onset.

Next: **Stage 2 — Pre-processing**, where the signals are cleaned, detrended and scaled before discovery.

STAGE 2

Pre-processing

Condition the signals so that discovery sees dynamics rather than drift and scale.

Causal discovery reads *time-lagged* relationships between signals, and two properties of the input decide whether it can. The series should be *stationary* — no drift and no permanent shift in level — and they should be on *comparable scales*, or a single high-magnitude tag will dominate every model. Stage 2 supplies six optional conditioning steps, always applied in the order 1 → 6.

The 30-second path

Expand **Step 4 · Detrending** and choose differencing → expand **Step 6 · Normalization** and choose standardization → **Apply pre-processing** → confirm the **Stationarity** table reads stationary for every variable.

Choosing what to process

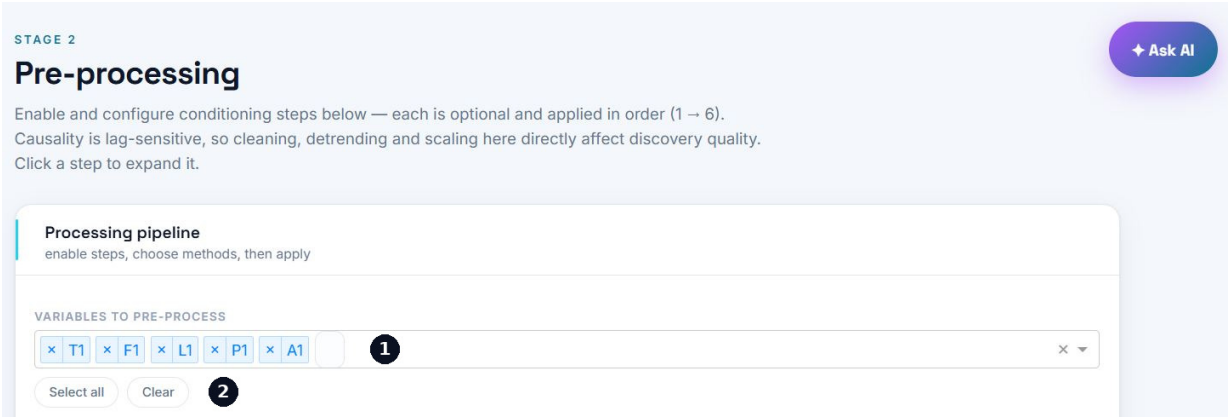

Figure 2.1: The page as it opens. Each step is optional; a step you never expand simply does not run.

1. **Variables to pre-process** — carried over from your Stage 1 selection. You can narrow it further here; only what is listed is conditioned and passed on to discovery.
2. **Select all / Clear** — shortcuts for the chip list.

Each step in one line, before you open any of them:

Step	Typical methods	Reach for it when
1 · Missing values	Linear interpolation, forward/back fill, mean, drop rows	The record has gaps and the time axis must stay continuous.
2 · Resampling	Keep every k-th sample	Sampling is much faster than the process dynamics.
3 · Outlier removal	Z-score clipping, IQR clipping	Spikes or instrument glitches would distort the model.
4 · Detrending	Linear detrend, first-order differencing, remove moving-average baseline	There is drift or a level shift — i.e. whenever the stationarity test fails.
5 · Noise filtering	Moving average, median filter, exponential smoothing	Measurement noise is visibly swamping the dynamics.
6 · Normalization	Standardization (z-score), min–max, robust	Almost always — tags rarely share a scale.

The six steps

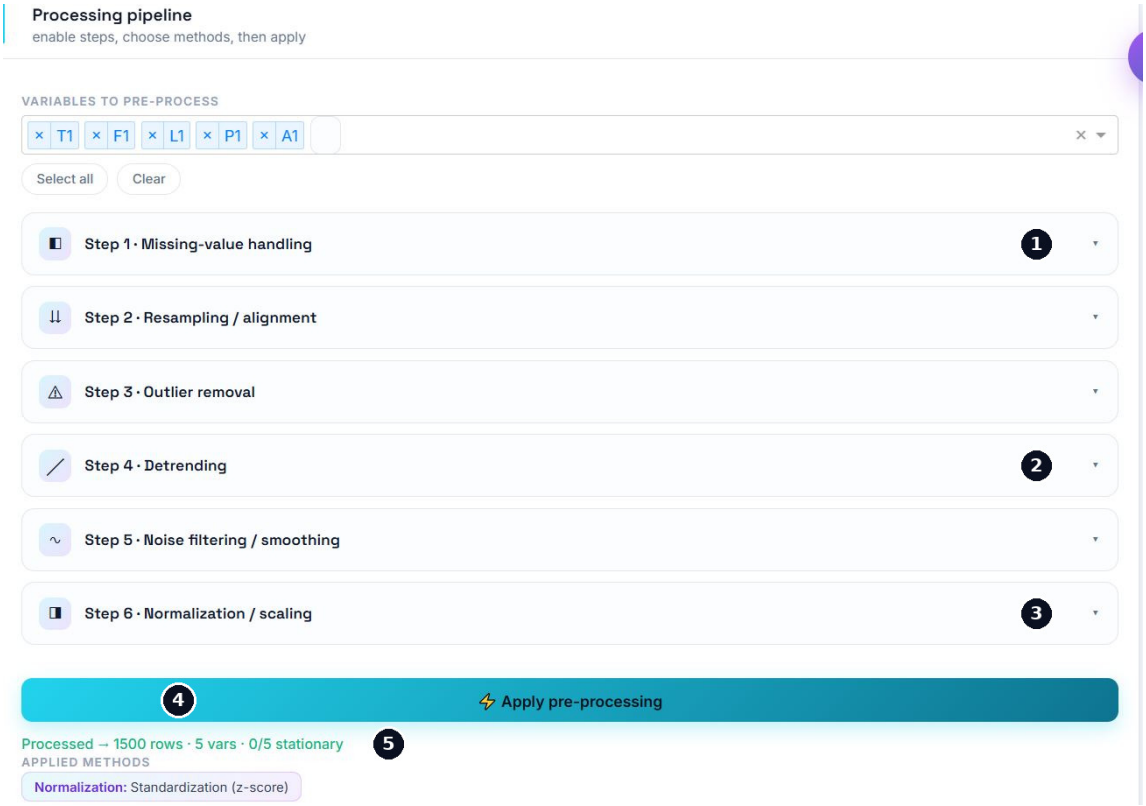

Figure 2.2: The pipeline collapsed. Click any row to expand it, enable the step and choose its method.

1. **A collapsed step** — open it to enable the step and pick a method; each method carries its own plain-language description inline.
2. **Step 4 · Detrending** — the step that decides stationarity, and the one most likely to change your result. First-order differencing is the workhorse.
3. **Step 6 · Normalization** — put every tag on a comparable scale. Standardization is the safe default; without any scaling, the highest-magnitude sensor can dominate the causal analysis on magnitude alone.
4. **Apply pre-processing** — nothing is conditioned until you press this, and it must be pressed again after every edit.
5. **Result line** — row count, variable count, and how many series passed the stationarity test. Beneath it, **Applied methods** chips list exactly what ran, which is your audit trail for the run.

Two settings that quietly damage discovery

- **Over-smoothing.** A long moving-average window spreads each sample into its neighbours, manufacturing lagged correlation that no physical coupling produced. Use the smallest window that removes the noise you actually have.
- **Resampling.** Keeping every k-th sample multiplies the physical meaning of one lag by k. If you resample here, revisit **Max lag** in Stage 3 accordingly.

Verifying the result

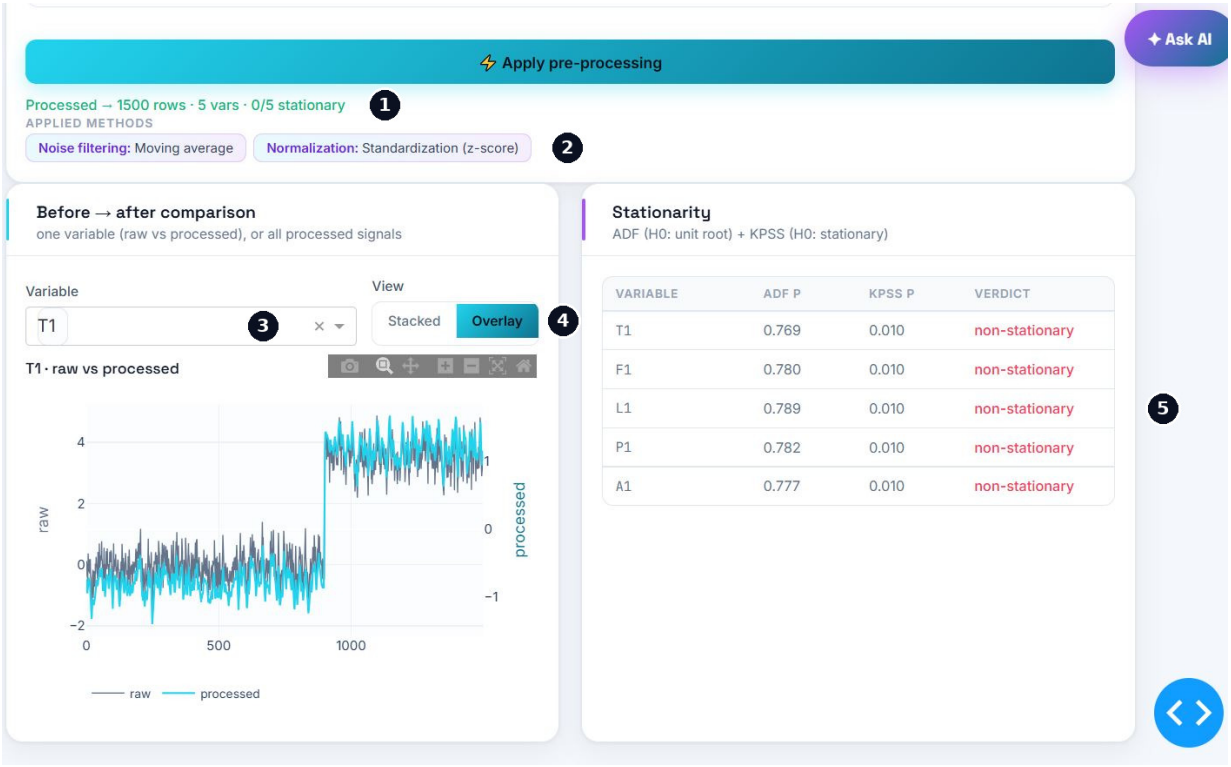

Figure 2.3: After applying smoothing and scaling only. The step change at sample 900 survives untouched — and the stationarity test says so.

1. **Result line** — here *1500 rows · 5 vars · 0/5 stationary*. The last figure is the one to read.
2. **Applied methods** — moving average and z-score. Neither addresses a shift in level, which is why the stationarity count is zero.
3. **Variable** — pick which tag the before/after plot shows.
4. **Stacked / Overlay** — overlay puts raw and processed on a shared time axis with separate vertical scales, which is the quickest way to confirm that conditioning preserved the *timing* of features. Stacked is easier for judging amplitude.
5. **Stationarity table** — ADF and KPSS p-values per variable, with a combined verdict.

Reading ADF and KPSS

The two tests are complementary, and their null hypotheses are opposites. ADF tests *"this series has a unit root"*, so a *small* p argues for stationarity. KPSS tests *"this series is stationary"*, so a *small* p argues against it. In Figure 2.3 both point the same way: ADF around 0.78 cannot reject a unit root, and KPSS at 0.010 rejects stationarity. KPSS p-values are clipped at 0.01 by the test's lookup table, so 0.010 means *"0.01 or less"*, not a precise value.

Why this run reads 0/5 — and how to fix it

The demonstration record contains a step disturbance at sample 900. A permanent change in mean level is non-stationary by construction, and neither of the applied methods removes it: smoothing only attenuates high-frequency noise, and standardization only re-centres and re-scales the whole series. Enable **Step 4** with first-order differencing and press **Apply pre-processing** again; the verdicts then flip to stationary. Note that the tests run over the whole record, both windows included — the fault is part of what they see.

If something looks wrong

What you see	What to do
Few or no variables stationary	Enable Step 4 and difference, then re-apply. If it still fails, the record may contain more than one operating point — shorten it or split the analysis.
Verdict reads *n/a*	The tests need `statsmodels`, a series longer than about 20 samples, and non-zero variance. A constant tag cannot be tested; drop it.
A message asking you to complete Stage 1	The scenario was never saved. Return to Stage 1 and press **Save scenario**.
Processed trace looks flat or featureless	The smoothing window is too large for the dynamics, or scaling was applied to a near-constant tag.
Row count dropped	Expected: differencing and row-dropping consume samples, and resampling divides them. Check the count is still comfortable for the method you plan to run.

Before you move on

- ☐ The result line is green, with the row and variable counts you expect.
- ☐ The stationarity table reads stationary for every variable — or you have a documented reason why it does not.
- ☐ In overlay, the processed trace still lines up with the features of the raw trace.

Next: **Stage 3 — Causal Discovery**, where the conditioned signals are turned into a directed, time-lagged graph.

STAGE 3

Causal Discovery

Infer the directed, lag-aware graph — and tell the tool what you already know.

This is the core of the toolbox. Everything downstream — monitoring, the root-cause ranking, the propagation path — is read off the graph produced here. Results are *method-dependent*: two algorithms resting on different assumptions will not return identical link sets, and agreement between them is the signal worth trusting.

The 30-second path

Skip domain knowledge on the first pass → family **Classic / statistical** → method **Conditional Granger (VAR)** → set **Max lag** comfortably above the slowest process delay → **Run discovery**. Then repeat with two or three methods from other families and compare.

Domain knowledge — optional, and the highest-value input you have

Purely data-driven discovery has no concept of piping, flow direction or control logic. You do. Entering what you already know is the single most effective way to improve accuracy on real plant data, because it stops the algorithm rediscovering — or contradicting — established process physics.

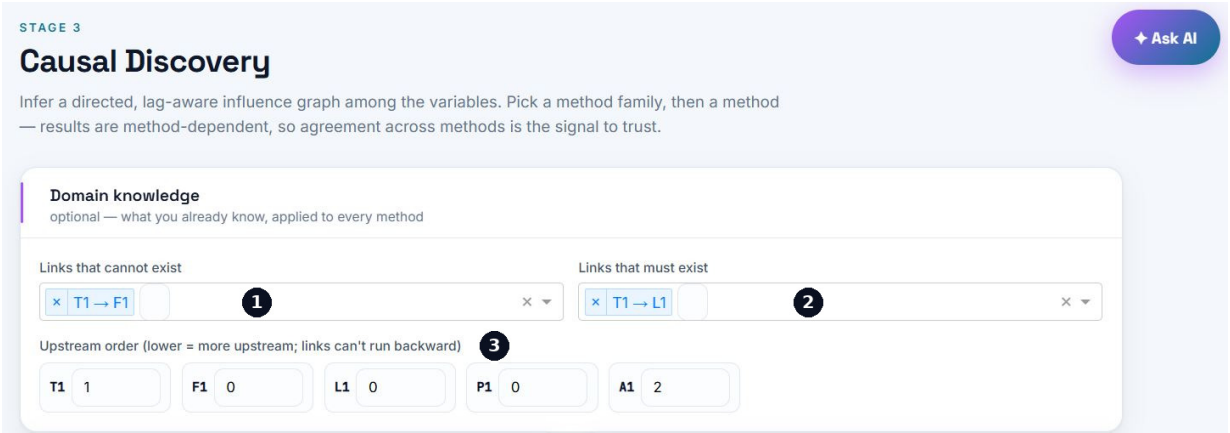

Figure 3.1: The three constraint types. Whatever you enter here is applied to *every* method, from the classic tests through to deep learning.

1. **Links that cannot exist** — ordered pairs that are physically impossible. The link is removed however strong the statistical evidence looks. Use it where a correlation is known to be non-causal: a downstream pressure cannot drive an upstream temperature, two sensors on isolated units cannot influence each other.

2. **Links that must exist** — pairs known to be coupled by design or by commissioning tests. If the method missed one it is added back with a representative strength. Data commonly fails to reveal a real link when the loop is tightly controlled (the controller suppresses exactly the variation the algorithm needs to see), when the true delay exceeds **Max lag**, or when sampling is too coarse.

3. **Upstream order** — an integer tier per variable: lower is further upstream, and any link running backwards against the order is removed. This is the most powerful of the three, because one tier assignment eliminates a whole class of reversed links at once. Derive the tiers from the process flow diagram — feed and utilities first, then each unit in sequence, analysers last. Variables sharing a tier are free to link in either direction.

Constraints are assertions of fact, not preferences

Forbidding a link you merely dislike, or requiring one you merely expect, will manufacture a graph that confirms your assumptions and hides what the data was trying to say. Encode only what process physics establishes.
The values in Figure 3.1 are there to show the controls, not to be copied: on the demonstration dataset `T1`→`F1` is a *genuine* edge, and `T1` is the true root, so forbidding that link and placing `T1` downstream of `F1` would be an error on this data. Note also that a required link running backwards against your tier order asserts two contradictory things at once — if you find yourself entering that combination, one of the two beliefs is wrong.

Constraints are not a cosmetic filter on the picture. They are applied to every method (for probabilistic methods a forbidden link has its probability, mean and standard deviation set to zero, and a required link is given probability 1); they are stored with the graph, so the Stage 6 reliability checks re-run under the same constraints and validate what you will actually use; and they are carried into the exported report so a reader can see which prior knowledge shaped the result.

Method, parameters and the graph

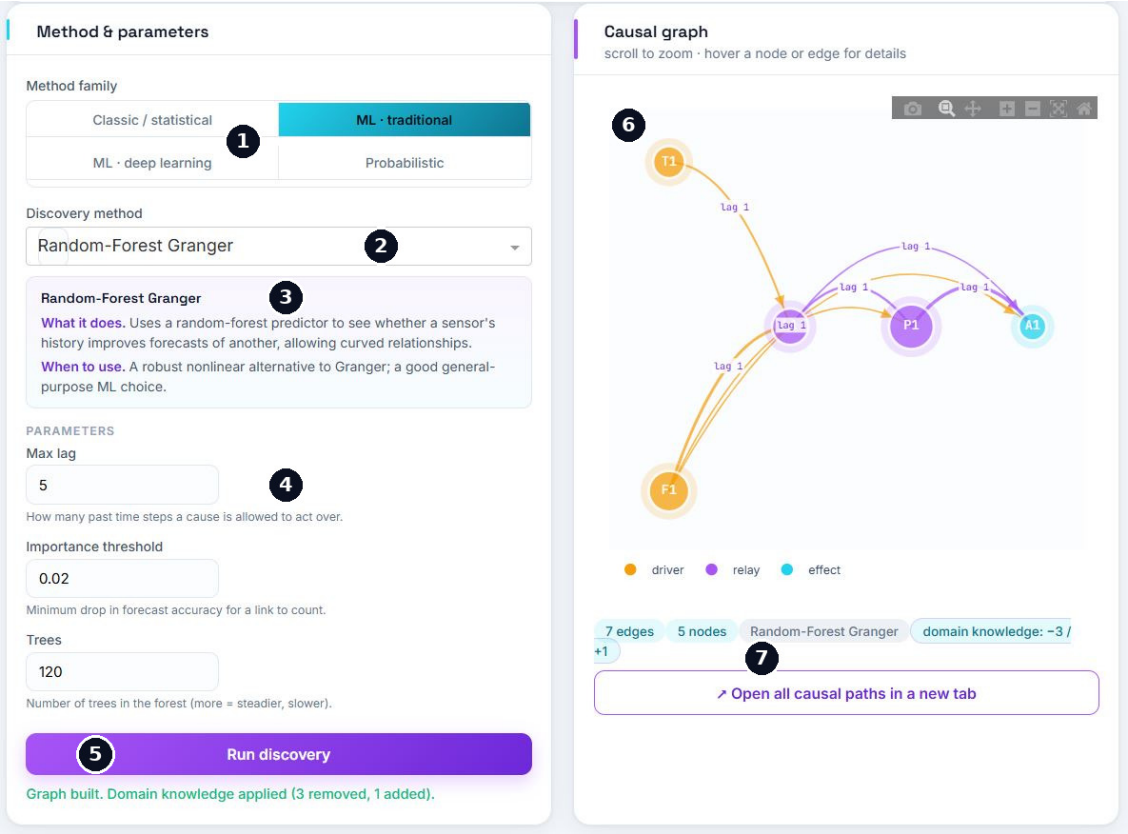

Figure 3.2: A completed run. The left card configures the method; the right card is the result.

1. **Method family** — picking a family filters the method list below. Nineteen algorithms are organised across the four.
2. **Discovery method** — the algorithm itself.
3. **Description** — every method explains, in plain language, what it does and when to reach for it. Read this before running something unfamiliar.
4. **Parameters** — the panel is *method-aware* and shows only the controls that apply, each

with a one-line explanation. Here: **Max lag** (how many past steps a cause may act over — set it comfortably above the slowest expected process delay), **Importance threshold** (the minimum drop in forecast accuracy for a link to count) and **Trees**. Switch method and the panel changes.

5. **Run discovery** — the green line beneath reports what happened, including any domain knowledge that was applied and how many links it removed and added. Check those counts match what you entered.

6. **Causal graph** — scroll to zoom; hover a node for how many variables it drives and is driven by, or an edge for its strength and lag. Arrow thickness encodes strength, the label on each arrow is the estimated delay, and node colour encodes structural role: **driver** (no incoming links — a source of influence), **relay** (both driven and driving) and **effect** (only driven — a terminal symptom). A node with no links at all is isolated.

7. **Summary pills and full paths** — edge and node counts, the method used, and the domain-knowledge tally. **Open all causal paths in a new tab** lists every causal chain in sequence with a table of all direct links, and offers HTML and CSV download — the practical way to read a graph too dense to follow on screen.

Family	Speed	Non-linear	Output	Best for
Classic / statistical	Fast	Only transfer entropy	Yes / no	First analysis and routine use. Conditional Granger (VAR) is the recommended default.
ML · traditional	Moderate	Yes	Yes / no	Suspected curvature. RandomForest Granger is the best general-purpose choice here.
ML · deep learning	Slow	Yes	Yes / no	Long records with complex or variable delays. Not worth it on a few hundred roughly linear samples.
Probabilistic	Moderate–slow	Partly	Probability	When you need a confidence per link rather than a yes/no.

The most reliable practice

No single method is authoritative. Run three or four from *different* families and trust the links they agree on — methods resting on different assumptions rarely make the same mistake. Every run you perform is collected in the report's methods-comparison table, which exists for exactly this purpose.

If something looks wrong

What you see	What to do
The graph is far too dense	Raise the importance threshold or lower α, prefer conditional over pairwise Granger, and add domain knowledge. Pairwise methods cannot separate direct from indirect links and return dense graphs on coupled plants.
The graph is empty	Loosen the threshold or α, raise **Max lag**, and check that pre-processing did not flatten the signals.
Links run backwards against process flow	Enter an upstream order. One tier assignment removes the whole class at once.
A method is greyed out or says *install to enable*	Its optional library is missing — `tigramite` for PCMCI, `lingam` for VarLiNGAM, `torch` for the deep-learning family.
A deep-learning method is very slow	Reduce epochs, shorten the window, or drop back to a traditional-ML method.

Before you move on

- ☐ The status line is green, and any domain-knowledge counts match what you entered.
- ☐ The edge count is plausible — neither a handful of links across dozens of tags, nor nearly every possible pair.
- ☐ The node roles are physically sensible: what you believe is upstream shows as a driver, and analysers show as effects.

Next: **Stage 4 — Monitoring**, which slides this same analysis across the record to see when the structure changed.

STAGE 4

Monitoring

A graph computed over one long window is a static picture. Slide the window, and you see when the structure changed.

Stage 4 moves an analysis window across the record, re-runs discovery inside each position, and tracks how much the resulting structure differs from the previous window. A structural shift is an early warning that something in the process is developing — often before any single tag has moved far enough to trip a conventional alarm. Monitoring reuses the method, parameters and domain-knowledge constraints from your last discovery run, so what it reports is consistent with the graph you have already inspected.

The 30-second path

Set **Window size** long enough for your method to be stable and **Step** as large as your time resolution allows → **Run monitoring** → read **Shifts flagged** and the sample index beneath, then find the responsible rows in the link-activity heat map.

Settings and summary

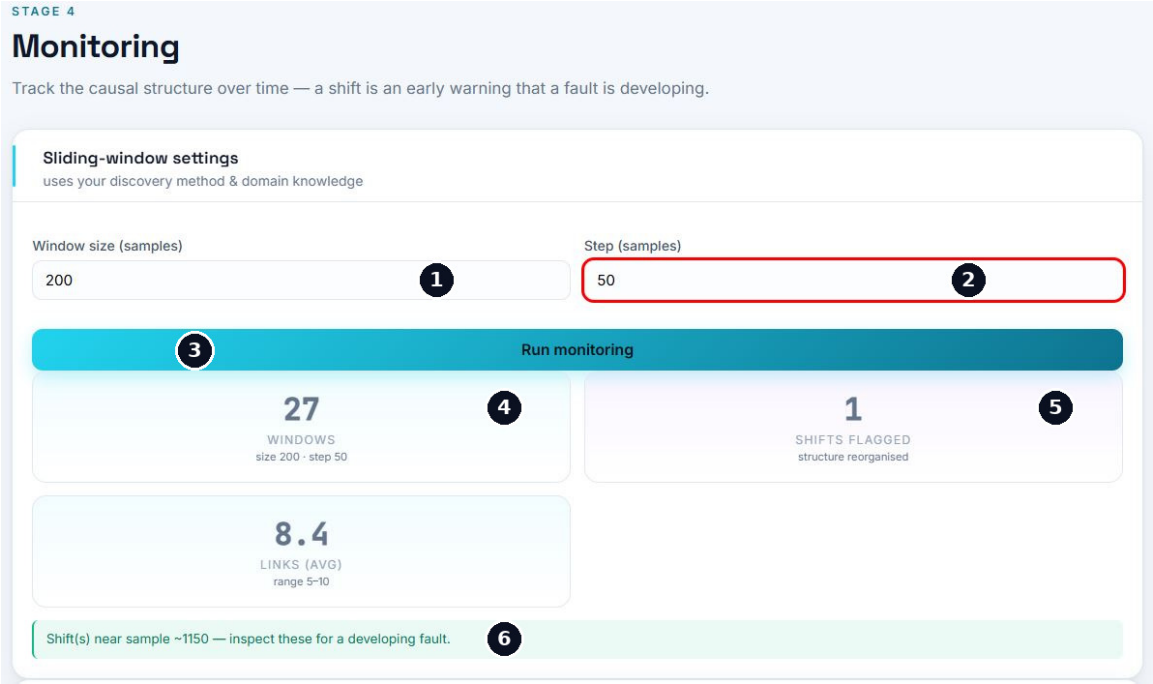

Figure 4.1: A completed monitoring run: 27 windows of 200 samples advancing in steps of 50.

1. **Window size** — samples per analysis window. It must be long enough for the chosen method to estimate a stable graph; a few hundred samples is a reasonable starting point, and heavier non-linear methods need more.

2. **Step** — how far the window advances each time. Smaller steps give finer time resolution at proportionally greater cost. The window count is roughly $(N - \text{size})/\text{step} + 1$, which is where the 27 here comes from.

3. **Run monitoring** — performs a *full discovery per window*, so this run is 27 complete analyses.

4. **Windows** and **Links (avg)** — the window count with the settings echoed back, and the

average number of links found per window with its range. A wide range (here 5–10) already tells you the structure is not settling to the same answer each time.

5. **Shifts flagged** — how many windows exceeded the automatic threshold for structural change.

6. **Verdict line** — names the sample index where a shift was flagged. Read it as a prompt to inspect, not as a diagnosis.

Cost

Every window is a full discovery run. With a deep-learning or bootstrap method and a small step this becomes expensive quickly, and the tool caps the analysis at 60 windows. For routine scanning, Conditional Granger is the fast choice.

Structural change over time

Figure 4.2: How much the graph moved between consecutive windows (top) and how many links each window contained (bottom).

1. **Structural change** — the number of links that differ from the previous window. The horizontal axis is the *centre* of each window, not its start.

2. **Threshold** — the automatic alarm level.

3. **Flagged window** — the star marks a window whose change exceeded the threshold.

4. **Links per window** — the raw edge count. A steady drift in this level is as meaningful as a spike above it: a graph that is quietly gaining or shedding links is also a changing process.

Read the baseline, not just the peak

In this run the structure never settles: 2–5 links change between consecutive windows even during normal operation, and the flagged peak of 6 is barely above that floor. That pattern is about the *configuration*, not the process — it usually means the window is too short for the chosen method to estimate stably, or that a heavy non-linear method is being asked to fit 200 samples. Lengthen the window, or re-run monitoring with Conditional Granger. A well-conditioned run shows change close to zero during normal operation and an unmistakable spike at onset.

The same caution applies to the flagged index. Here the shift is reported near sample 1150, while the demonstration fault is injected at sample 900 — and a window centred on 1150 spans 1050–1250, entirely after the step. Treat the flagged sample as approximate, and cross-check it against the Stage 1 preview and the Stage 5 result rather than reporting it as the fault time.

Link activity

Figure 4.3: One row per candidate link, one column per window. This is where you find out *which* connections moved.

1. **Rows** — every candidate link, named source → target. A row that stays blank across the whole record is a link the method essentially never found; a row that switches on partway through is the interesting case.

2. **Cells** — link strength in that window. Scan horizontally for connections that appear, fade or strengthen, and line those transitions up against the flagged sample from Figure 4.2.

3. **Colour scale** — shared across all rows, so intensities are comparable between links as well as across time.

If something looks wrong

What you see	What to do
Only a handful of windows	The window size or step is large relative to the record. Reduce one or both, but keep the window long enough for the method.
The run is very slow, or the window count is capped	The 60-window cap has been reached, or the method is heavy. Increase the step, or re-run with Conditional Granger.
Structural change is noisy everywhere	The window is too short for the method to estimate stably. Lengthen it or use a lighter method before interpreting any spike.
No shift flagged, but you know a fault occurred	A gradual change may never cross the threshold. Read the heat map rows directly rather than relying on the alarm.
The heat map is almost empty	The Stage 3 threshold is too strict for these shorter windows — each window has far less data than the full-record run did.

Before you move on

☐ Structural change is quiet during normal operation, so a spike means something.

☐ Any flagged sample is consistent with what the Stage 1 preview shows.

☐ You can name the specific links that changed, from the heat map.

Next: **Stage 5 — Root-Cause Identification**, which ranks candidates and traces the propagation path.

STAGE 5

Root-Cause Identification

Separate the originating fault from its downstream symptoms, and trace how it spread.

Stage 5 combines two things you already have — the causal graph from Stage 3 and the record of which variables went anomalous — and answers the operational question directly: *where did it start, what did it pass through, and how long did each step take?*

The 30-second path

Analyse root cause → read the top candidate and its score → follow the propagation chains → read the plain-language summary into your shift log.

Ranking and propagation

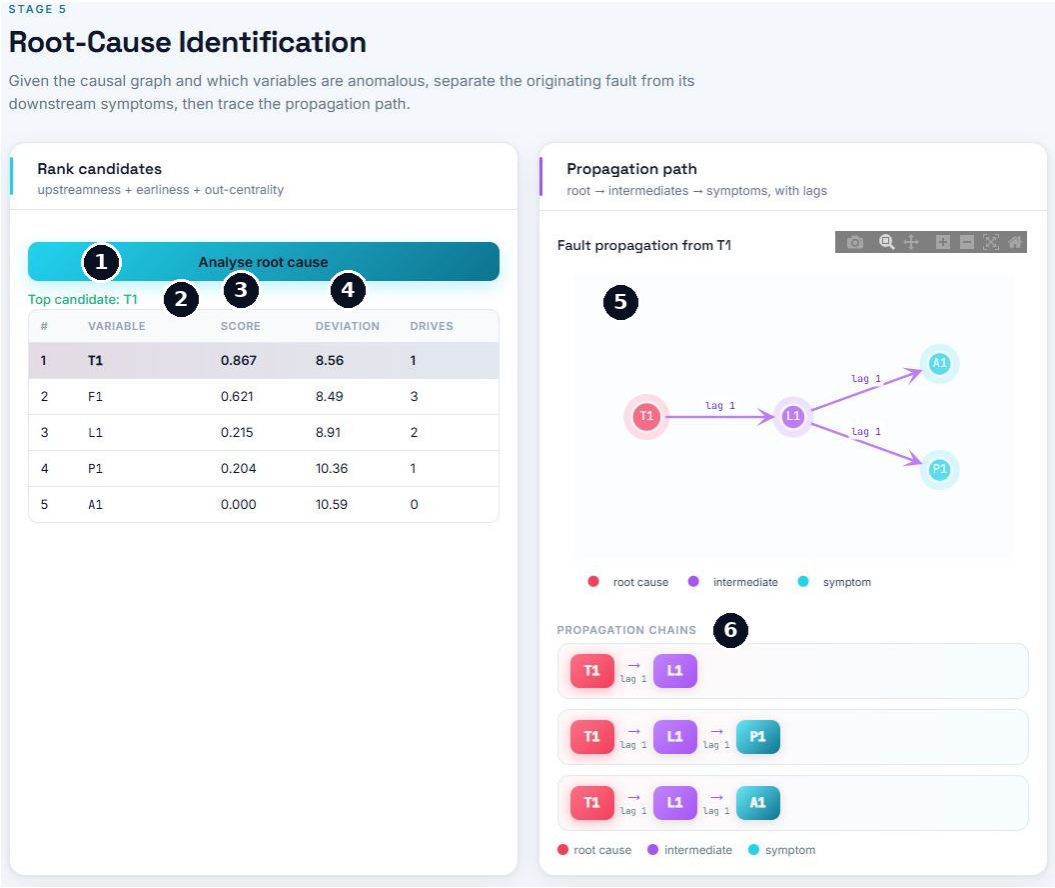

Figure 5.1: The ranked candidates on the left, the traced propagation on the right.

1. **Analyse root cause** — runs the ranking against the current graph. If nothing happens, no graph exists yet; return to Stage 3.
2. **Top candidate** — the tool's answer, repeated above the table and highlighted in the first row.
3. **Score** — the combined ranking figure, from 0 to 1. The card header names what goes into it: *upstreamness*, *earliness* and *out-centrality*.
4. **Deviation and Drives** — how far the sensor departed from its baseline, and how many

variables it drives in the graph. Deviation is shown as context; it is not what decides the ranking, for the reason in the box below.

5 **Propagation path** — the root in red, intermediates in violet, terminal symptoms in cyan, with the estimated delay on each step.

6 **Propagation chains** — the same information as an ordered list, one row per route from the root. This is the form to copy into a handover note.

The biggest deviation is usually not the cause

Read the two numeric columns in Figure 5.1 against each other. `T1` is ranked first with the *smallest* deviation in the table (8.56), while `A1` has the largest (10.59) and scores zero. That is not a quirk of this dataset — it is the normal pattern. Effects accumulate as a disturbance travels downstream, so the most visibly disturbed sensor is usually a late symptom, and the sensor that actually failed may barely stand out. This is precisely why causal structure, not deviation magnitude, drives the diagnosis.

The path inherits your graph — and your constraints

The propagation here runs `T1`→`L1`→ {`P1`, `A1`}, and `F1` appears in no chain at all, even though the summary lists it among the downstream symptoms. That is a direct consequence of the domain knowledge entered in Stage 3, where `T1`→`F1` was forbidden and `T1`→`L1` required: the constrained graph has no route through `F1`, so no chain can pass through it.
Whenever a variable you expect to be involved is missing from the propagation, check your constraints before questioning the process. The path can only be as good as the graph it is read from.

What the scores mean

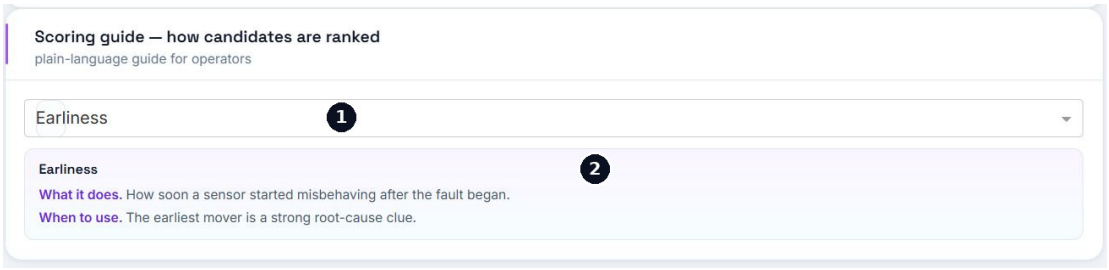

Figure 5.2: Every scoring factor explains itself in plain language — built for handing the screen to someone who was not part of the analysis.

1 **Factor selector** — pick any factor used in the ranking.

2 **Description** — what it measures and why it is evidence.

Factor	What it measures
Earliness	How soon the sensor started misbehaving after the fault began. The earliest mover is a strong root-cause clue.
Upstreamness	How far upstream it sits in the causal graph.
Out-centrality	How many other variables it drives — the **Drives** column.
Deviation	How far it departed from baseline behaviour. Useful context, and a deliberate counterweight to intuition rather than a ranking driver.

The plain-language summary

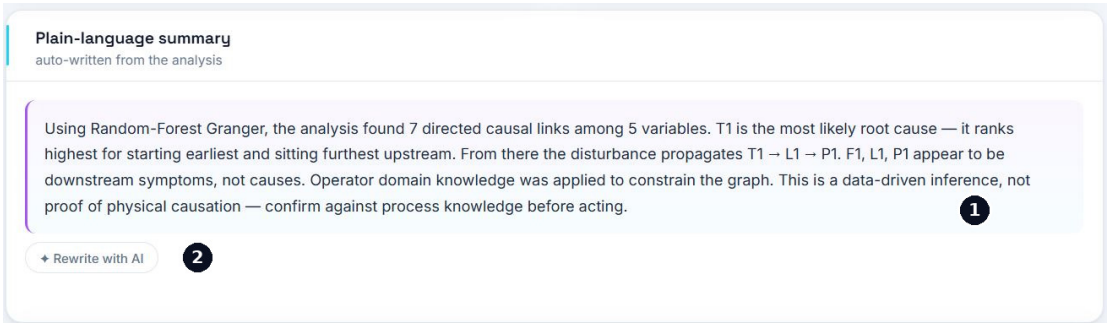

Figure 5.3: The narrative the tool writes about its own findings, refreshed whenever you re-run a stage.

1. **The summary** — method and link count, the root cause and why it ranked first, the propagation chain, which variables are symptoms, the Stage 6 reliability verdict once run, any domain knowledge applied, and a standing caution that this is inference and not proof.

2. **Rewrite with AI** — passes the summary to whichever model you have configured and asks for a more fluent three-to-five-sentence briefing.

Why this text is trustworthy — and where that stops

The summary is generated *deterministically* from the stored results. It is template-driven, so it needs no API key, no internet and no external service; it cannot hallucinate, because every number in it is read straight from the computed results; and it is reproducible, so the same analysis always yields the same wording — which matters for an auditable record.
None of that applies to the AI rewrite. A language model can subtly shift emphasis or overstate confidence, so read the rewritten version against the deterministic one — which remains available and unchanged — before putting it into a formal record.

If something looks wrong

What you see	What to do
The top candidate is a known symptom	The graph has the direction wrong somewhere. Return to Stage 3 and set an upstream order, or forbid the reversed links.
Every score is near zero, or several tie	The graph is too sparse to separate candidates. Loosen the Stage 3 threshold or raise **Max lag**.
A variable you expect is missing from the propagation	It is not reachable from the root in the current graph. Check forbidden links first, then **Max lag**.
Analyse root cause produces nothing	No causal graph exists yet. Run Stage 3 first.
The result looks convincing	Confirm it before acting. A high score says the graph is internally consistent, not that the graph is right — that is what Stage 6 is for.

Before you move on

- ☐ The top candidate is physically plausible as an origin, not merely the most disturbed tag.
- ☐ The propagation chains correspond to real process connections.
- ☐ Any domain knowledge noted in the summary is what you intended to impose.

Next: **Stage 6 — Evaluation & Report**, which tests how far this result can be trusted and exports it.

STAGE 6

Evaluation & Report

Check how far the graph can be trusted, then export it.

On a real plant the true causal graph is unknown, so precision and recall cannot be computed. This is not a minor inconvenience — it is the central difficulty of applied causal discovery, and Stage 6 is built around it. The left card works *without* any reference and is your primary quality evidence. The right card applies only where a reference exists: the synthetic demo, or a plant whose topology you know from a P&ID.

The 30-second path

Run reliability checks → read **Predictive gain** first, then the other three → on the demo, also **Score vs ground truth** → export **HTML** or **PDF**.

Reliability, with and without a reference

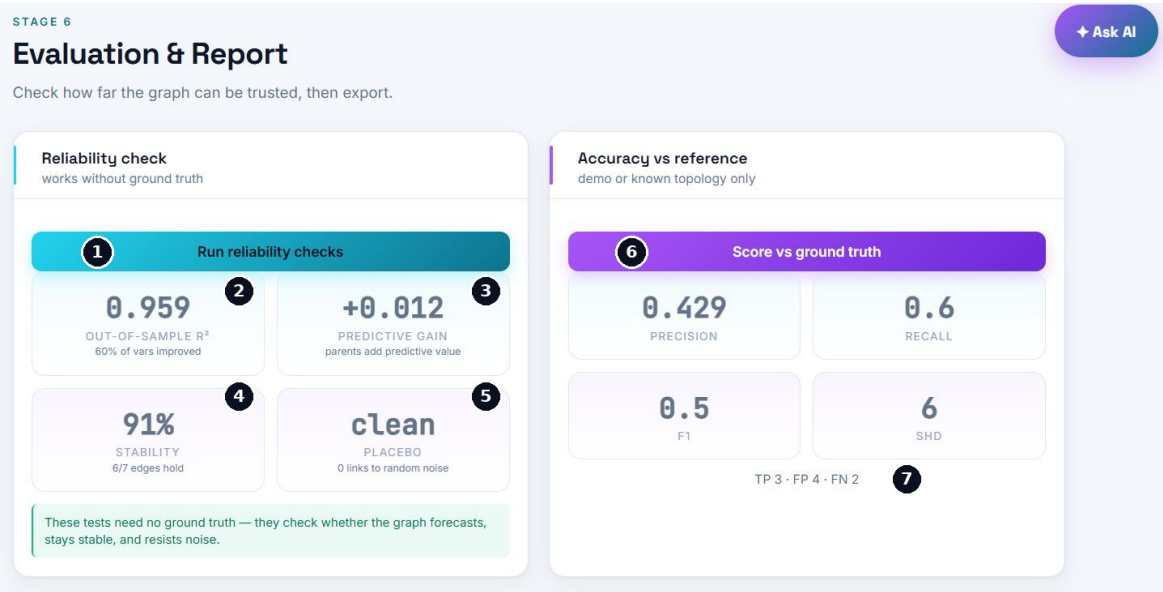

Figure 6.1: Both cards populated. The left four tiles are available on any dataset; the right four require a known structure.

1. **Run reliability checks** — re-runs your method under resampling and noise injection, so it takes roughly as long as the original discovery.

2. **Out-of-sample R^2** — how well each variable is predicted on data the model never saw, using its discovered parents. Judge it relative to the process: a noisy analyser will never match a smooth temperature.

3. **Predictive gain** — the improvement over predicting each variable from *its own past alone*. This is the single most important number on the page, and the one to read first.

4. **Stability** — how often each discovered link reappears when the data is resampled in contiguous blocks. Above roughly 70 % is solid.

5. **Placebo** — a column of pure noise is injected and discovery re-run. *Clean* is the expected result; any link to the placebo is a provable false positive.

6. **Score vs ground truth** — precision, recall, F1 and SHD against the known structure. SHD counts the edits — add, remove or reverse — needed to turn your graph into the true one, and unlike F1 it penalises a reversed arrow, which matters greatly for root-cause work.

7. **TP · FP · FN** — the raw counts behind those four figures, and usually the fastest way to see what actually went wrong.

This run is a worked example of the main trap

Read the left card in order. Out-of-sample R^2 is a very healthy 0.959 — but predictive gain is +0.012, essentially zero, and only 60 % of variables improved at all. Those two facts together say that almost all of that 0.959 comes from each variable's *own past*: these are smooth, autocorrelated process signals, and the discovered parents are adding next to nothing on top. A causal parent must carry information about its child that the child's own history does not already contain; when it does not, the links are decorative.
Stability of 91 % and a clean placebo do not rescue that. They say the method returns the *same* links each time and does not chase noise — consistency, not correctness. The demo is the one place you can check that distinction directly, and the right-hand card duly reports precision 0.429: of seven reported links, four are false positives.

Where those errors came from

The domain knowledge entered in Stage 3 accounts for two of them by construction. `T1→L1` was *required* but is not in the true structure — a guaranteed false positive. `T1→ F1` was *forbidden* but is a true edge — a guaranteed false negative. Two of the six SHD edits were bought with prior knowledge that was not correct.
This is exactly what the manual means by using constraints honestly. Constraints propagate: they shape the graph in Stage 3, the propagation path in Stage 5, and the scores here — and because the reliability checks re-run the method under the same constraints, a confidently wrong assertion will look stable.

Pattern	What it means	What to do
High gain, high stability, clean placebo	The graph is trustworthy.	Proceed to root-cause analysis with confidence.
High gain, low stability	The structure is real, the specific links uncertain.	Use a probabilistic method to quantify which ones.
Low gain, high stability	Consistently the same links, carrying little predictive information.	Change method or re-check stationarity before trusting the graph.
Placebo not clean	Settings are too permissive.	Tighten the threshold before interpreting anything else.

Do not report the accuracy metrics on real plant data

Without a known reference graph, precision, recall, F1 and SHD cannot be computed, and any number claiming otherwise is meaningless. Use the four reliability checks instead. If you know part of the topology from a P&ID you have a partial reference — say so explicitly when you report the figures.

The report

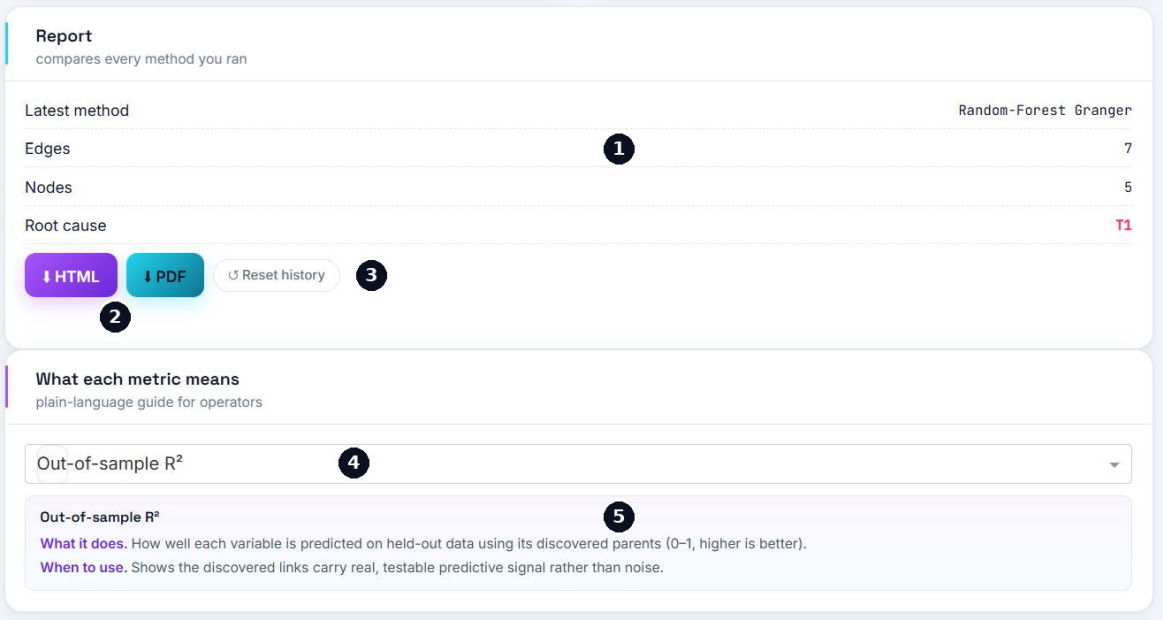

Figure 6.2: The export card and the metric glossary that ships with it.

1. **Run summary** — the latest method, edge and node counts, and the identified root cause. The report itself covers *every* method you ran, not just this one.

2. **HTML and PDF** — HTML is self-contained with an interactive causal graph; PDF is print-ready with a natively drawn graph. Both contain the narrative summary, the configuration, the graph, the causal paths in sequence, any applied domain knowledge, per-edge uncertainty for probabilistic methods, the methods comparison, every metric with its plain-language description, the root-cause ranking and the traced propagation.

3. **Reset history** — clears the accumulated record of methods run. Use it when starting a genuinely new investigation, not between methods on the same one.

4. **Metric selector** — every metric used anywhere in the toolbox.

5. **Description** — what it measures and when it is the right thing to look at. The same descriptions are printed in the exported report, so a reader who was not present can follow it unaided.

Complete the run before exporting

The metrics section is populated from what you have actually computed. Run the reliability checks — and, on the demo, the structural evaluation — before exporting, or the report will have gaps where its strongest evidence should be.

If something looks wrong

What you see	What to do
Predictive gain near zero	The discovered links may be spurious. Try another method, re-check stationarity in Stage 2, or lengthen the record — before acting on any root cause.
Gain is zero for one variable only	Expected if that variable is the root: it has no parents, so there is nothing to improve its forecast. Check *which* variable before treating it as a problem.
The placebo is not clean	Settings are too permissive. Lower α or raise the importance threshold and re-run discovery.
Structural accuracy is unavailable	Expected on real data — no reference graph exists. Use the reliability checks.
The report is missing sections	Those stages were never run. Return and complete them, then export again.

Before you sign it off

- ☐ Predictive gain is positive across most variables — not just a high R^2.
- ☐ Stability is comfortable and the placebo is clean.
- ☐ Any structural accuracy figures are accompanied by a statement of where the reference came from.
- ☐ The exported report contains every stage you intend to stand behind.

OPTIONAL

The AI Assistant

Ask questions about your own run, in ordinary language.

Every page carries a floating ★ **Ask AI** button in the top-right corner. It opens a chat panel connected to a language model that has been given a summary of your current analysis, so you can ask about *your* results rather than about causal discovery in general.

Entirely optional

The assistant is a convenience layer, not a dependency. Every analytical capability — including the plain-language summary of Stage 5 — works fully without it. If you never configure a provider, nothing else in the toolbox is affected.

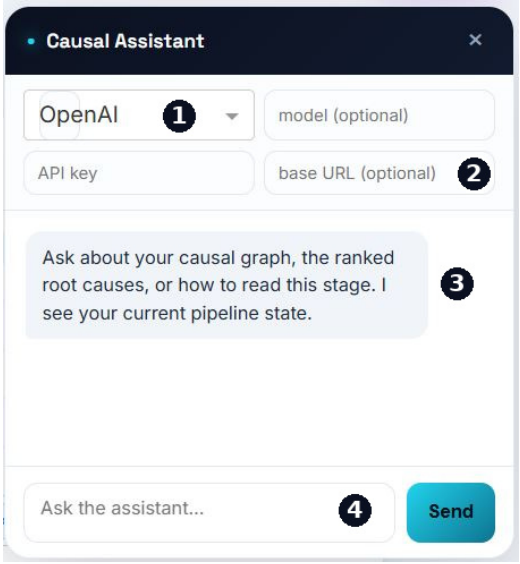

Figure 7.1: The assistant panel, before any provider has been configured.

1. **Provider** — OpenAI, Anthropic, Ollama or a custom endpoint.
2. **Credentials** — an API key, and optionally a model name and base URL. The base URL is for an in-house or corporate gateway exposing an OpenAI-compatible API; with Ollama it points at your local server.
3. **Conversation** — the assistant states up front that it can see your current pipeline state.
4. **Ask box** — one focused question at a time works far better than several at once; the context it receives is compact.

Data confidentiality

With a cloud provider, the context below — variable names, the method used, edge counts and the root-cause ranking — is transmitted to that provider. Raw process data is never uploaded, but *tag names alone may be sensitive*. If your site restricts external data transfer, use Ollama to keep everything on your own machine, or leave the assistant unconfigured.

What it can and cannot do

Before each question the toolbox builds a compact description of your run: the selected variables and both windows, the pre-processing applied, the discovery method and how many links it

found, whether domain-knowledge constraints were used, and the top root-cause candidates with their scores. It is refreshed automatically as you advance through the stages.

Good use	**Example question**
Understanding a result	Why is `T1` ranked above `P1` when `P1` deviates more?
Choosing a method	My data looks non-linear and I have 5000 samples — what should I try next?
Interpreting a metric	My stability is 45 %. Is that acceptable, and what should I change?
Deciding on constraints	The graph says the analyser drives feed temperature. Should I forbid that link?
Drafting a write-up	Turn these findings into three sentences for a shift handover.

Where not to rely on it

- It sees a *summary* of the analysis, not the raw data. It cannot recompute anything or verify a number for you.
- Language models state incorrect things fluently. Treat its answers as suggestions from a knowledgeable colleague — helpful, not authoritative.
- It knows nothing about your plant beyond that summary: not your equipment, your control strategy or your operating limits.
- Never let it substitute for the numbers. The Stage 6 reliability checks are evidence; the assistant's opinion is not.

Ask it to *explain* rather than to *decide*. *What does this metric mean for my case* works far better than *what should I do*.

APPENDIX

The Developer Panel

The floating controls in the bottom-right corner are not part of the toolbox.

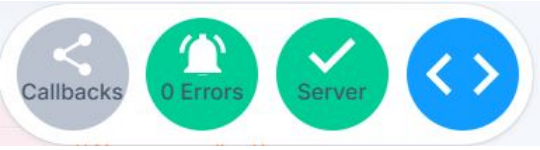

Figure A.1: The panel expanded. The blue <> circle on its own is what you see until you click it.

This cluster belongs to Dash, the framework the toolbox is built on, and appears only when the application is started with debugging enabled. It displays nothing about your analysis and cannot change a result.

- **< >** — opens and closes the panel. Collapsed, this blue circle is the only thing visible.
- **Callbacks** — a dependency graph of the application's internal callbacks. Of interest when developing the tool, not when using it.
- **0 Errors** — how many internal errors have occurred in this browser session. Click it to read the traceback.
- **Server** — the development server's status. A green tick means it is running and responding.

The one time it is useful to you

If the interface misbehaves — a button does nothing, a plot fails to appear — open the panel and check the error count before restarting. Copying the traceback, together with the steps that produced it, turns *"it broke"* into something that can actually be fixed.

Turn it off for operator use

The panel is present because the application was launched in debug mode (`app.run(debug=True)`). For anything an operator touches, start the app with debugging disabled — or serve it through a production WSGI server such as Gunicorn or Waitress — and these controls will not appear at all. Debug mode also reloads the application when files change and exposes internal detail in error pages, neither of which belongs on a plant terminal.

One last thing

Causal discovery from observational data yields *plausible causal structure*, not proof. Its conclusions are conditional on the variables you recorded, the pre-processing you chose and the assumptions of the algorithm. An unmeasured common driver can manufacture a link between two variables that do not influence each other; a delay longer than **Max lag** will be missed entirely; and a tightly controlled loop can mask causal direction because feedback keeps the variables in step.
Corroborate against process knowledge, and where the decision matters, against a controlled test. For the reasoning behind any of this — method assumptions, metric definitions, interpretation limits — the *Operator and Engineer Manual* is the authority.